

12 Factor-App

Présentation

Le principe des 12-Factor App^[1] décrit comment concevoir une application cloud-native propre, portable, scalable et maintenable. L'idée centrale est simple : l'application doit être facile à déployer, facile à faire évoluer, indépendante de son environnement, stateless quand c'est nécessaire, et compatible avec l'automatisation.^[2]



1. Codebase

→ Une application doit avoir **une seule base de code**.

Cette base de code est versionnée dans un système de contrôle de version comme Git, puis utilisée pour plusieurs environnements de déploiement : développement, QA, staging, production.

Idée clé :

- Une application = un dépôt principal.
- Si le système est trop grand, il faut le découper en plusieurs applications distinctes, chacune avec sa propre codebase.

Pourquoi c'est important :

- Cela garantit la cohérence du code, simplifie la collaboration entre développeurs et évite les divergences entre environnements.

2. Dependencies

→ Toutes les dépendances de l'application doivent être **déclarées explicitement** et **isolées**.

Idée clé :

- L'application ne doit jamais supposer que certaines bibliothèques ou outils existent déjà sur la machine d'exécution.
- Elle doit déclarer précisément ce dont elle a besoin, y compris les **versions exactes**.

Pourquoi c'est important :

- Cela rend l'application reproductible, portable et moins sujette aux erreurs liées à l'environnement.

Exemple :

- Utiliser un fichier comme `package.json`, `requirements.txt`, `pom.xml`, etc.
-

3. Configuration

→ La configuration d'une application correspond aux valeurs qui changent selon l'environnement : base de données, URLs, identifiants, SMTP, clés API, etc.

Idée clé :

- La configuration doit être stockée **hors du code**, généralement dans des **variables d'environnement**.

Pourquoi c'est important :

- Le code reste identique entre les environnements, seule la configuration change. Cela évite de mélanger logique métier et paramètres de déploiement.
-

4. Backing services

→ Les backing services sont tous les services externes accessibles via le réseau : base de données, cache distribué, serveur mail, file de messages, stockage, etc.

Idée clé :

- L'application doit considérer ces services comme des **ressources attachables/détachables**, sans dépendre de leur emplacement ou de leur nature exacte.

Pourquoi c'est important :

- On peut remplacer un service sans changer le code. Par exemple, remplacer une base de données défaillante par une autre à partir d'une sauvegarde.
-

5. Build, release, run

→ Les étapes **build**, **release** et **run** doivent être **strictement séparées**.

Build : transformation du code en artefact exécutable

Release : combinaison de l'artefact avec la configuration

Run : exécution réelle de l'application

Pourquoi c'est important :

- Cette séparation rend les déploiements plus sûrs, traçables et reproductibles.
-

6. Processes

→ L'application doit être conçue comme un ensemble de **processus stateless** et **share-nothing**.

Idée clé :

- Un processus ne doit pas dépendre de sa mémoire locale ou de son disque local pour conserver un état entre deux requêtes.

Pourquoi c'est important :

- Cela facilite la scalabilité, la tolérance aux pannes et le redémarrage des processus.

Bon exemple :

- Une image téléversée est stockée dans un stockage objet, les métadonnées dans une base de données, et le traitement est envoyé dans une queue.
- → N'importe quel worker peut reprendre le travail.

Mauvais exemple :

- Sauvegarder localement un fichier dans le disque d'un worker puis supposer qu'un autre traitement retrouvera ce fichier au même endroit plus tard.

Pour l'authentification et l'autorisation :

- L'état de session ou les droits ne doivent pas être conservés uniquement en mémoire dans une instance de l'application. Ils doivent être dans Redis, une base de données, ou dans un token signé.
-

7. Port binding

→ L'application doit être **autonome** et exposer ses services via un **port défini**.

Idée clé :

- L'application s'exécute elle-même et écoute sur un port donné, plutôt que de dépendre d'un serveur web externe injecté de manière implicite.

Pourquoi c'est important :

- Cela améliore la portabilité et simplifie le déploiement sur différents environnements.

Exemple :

- Une application Node.js lit le port via une variable d'environnement au lieu de le coder en dur.
-

8. Concurrency

→ La montée en charge doit se faire en **ajoutant des processus**, pas en rendant une seule instance toujours plus grosse.

Idée clé :

- On sépare les responsabilités par types de processus.

Exemple classique :

- un processus `web` pour les requêtes HTTP
- un processus `worker` pour les tâches de fond

Pourquoi c'est important :

Cela améliore la scalabilité, la réactivité et la résilience.

Exemple :

Dans un e-commerce :

- le web reçoit la commande
- il enregistre les données et envoie un job dans une queue
- un worker traite le paiement, l'email de confirmation et le stock

Si le trafic augmente, on lance plus de workers ou plus d'instances web.

9. Disposability

→ Les processus doivent pouvoir être **démarrés rapidement** et **arrêtés proprement**.

Idée clé :

- L'application doit supporter le fait que les processus soient créés et détruits fréquemment.

Pourquoi c'est important :

- C'est indispensable dans les environnements conteneurisés et orchestrés, comme Kubernetes.

Concrètement :

- startup rapide
 - arrêt gracieux
 - pas de corruption d'état
 - pas de perte de données en cours
-

10. Development/production parity

→ Il faut réduire au maximum l'écart entre les environnements de **développement** et de **production**.

Différences à éviter :

- outils différents
- versions différentes
- services externes différents
- délais trop longs entre code et mise en prod
- équipes de dev et d'exploitation trop séparées

Pourquoi c'est important :

- Plus l'écart est grand, plus on découvre tard des bugs ou comportements inattendus.

Bonnes pratiques :

- Docker
 - Infrastructure as Code
 - gestion cohérente des configurations
 - mêmes dépendances et mêmes services autant que possible
-

11. Logs

→ L'application ne doit pas gérer elle-même le stockage ou la rotation des logs.

Idée clé :

- Chaque processus doit écrire ses logs sur **stdout** et **stderr**, comme un flux d'événements.

Pourquoi c'est important :

- L'environnement d'exécution se charge ensuite de collecter, router, agréger et stocker les logs.

Exemples d'outils :

- ELK Stack
- AWS CloudWatch Logs
- autres solutions centralisées

Bonne pratique :

- Produire des logs structurés pour faciliter la recherche et l'analyse.
-

12. Administrative processes

→ Les tâches d'administration doivent être exécutées comme des **processus ponctuels séparés**.

Exemples :

- migration de base de données
- nettoyage de données
- génération de rapports
- scripts d'analyse

Idée clé :

- Il ne faut pas mélanger ces tâches avec le cycle normal de l'application.

Pourquoi c'est important :

- Cela améliore l'isolation, la maintenabilité et la sécurité.
-
-

1. <https://12factor.net/> ↩

2. <https://www.softfluent.fr/blog/the-12-factor-app-sont-ils-toujours-dactualite/> ↩